

1. join() Method (Array → String)

Syntax

```
array.join(separator)
```

- separator is optional.
- Default separator is , (comma).

2. split() Method (String → Array)

Syntax

```
string.split(separator)
```

Limit the Number of Elements: → `string.split(separator,limit)`

Difference Between join() and split()

Feature	join()	split()
Works On	Array	String
Returns	String	Array
Purpose	Combine array elements	Break string into array
Syntax	<code>arr.join(",")</code>	<code>str.split(",")</code>

Quick Revision

Method	Input	Output
join()	Array	String
split()	String	Array
reverse()	Array	Array
sort()	Array	Array
push()	Array	Array (modified)
pop()	Array	Removed element
shift()	Array	Removed first element
unshift()	Array	Array (modified)

1. includes() Method

Purpose

Checks whether an array contains a specific value.

Case-sensitive

Syntax

```
array.includes(value)
```

Returns:

- true → if the value exists.
- false → if the value does not exist.

2. find() Method

Purpose

Returns the **first element** that satisfies a condition.

Syntax

```
array.find(function(element) {  
    return condition;  
});
```

Returns:

- Matching element
- undefined if no match is found

includes() vs find()

Feature	includes()	find()
Purpose	Checks if a value exists	Finds the first element matching a condition
Returns	true or false	Element or undefined
Uses Callback?	No	Yes
Works Well With	Primitive values	Objects and custom conditions

Quick Revision

Method	Returns	Callback Required
includes()	true / false	No
find()	First matching element	Yes
findIndex()	Index of first matching element	Yes
filter()	Array of all matching elements	Yes

→ In class, Methods are not compulsory to create. (S1.rollno => to access the value of rollno)

→ Template Literal

1. Methods in a Class are Not Compulsory

In JavaScript, a class can contain:

- Properties (variables)
- Methods (functions)

A class **does not have to contain methods**. You can create a class with only properties.

Example

```
class Student {  
  rollno = 101;  
  name = "Prince";  
}  
  
let s1 = new Student();  
  
console.log(s1.rollno);  
console.log(s1.name);
```

Output

```
101  
  
Prince
```

Explanation

- Student is a class.

- rollno and name are properties.
 - No methods are created.
 - s1.rollno is used to access the value of rollno.
 - s1.name is used to access the value of name.
-

2. Accessing Properties

Use the **dot (.) operator** to access object properties.

Syntax

objectName.propertyName

Example

```
class Student {  
    rollno = 101;  
    name = "Prince";  
}  
  
let s1 = new Student();  
  
console.log(s1.rollno);  
console.log(s1.name);
```

Output

```
101  
Prince
```

3. Template Literals

A **Template Literal** is a string written using **backticks (`)** instead of single (') or double (") quotes.

It allows:

- Variable interpolation using `${}`
- Multi-line strings

Syntax

```
`Text ${variable}`
```

Example 1: Variable Interpolation

```
let name = "Prince";  
  
let age = 21;  
  
console.log(`My name is ${name} and I am ${age} years old.`);
```

Output

My name is Prince and I am 21 years old.

forEach() Method in JavaScript

The **forEach()** method is used to **iterate (loop) through each element of an array**.

- Executes a function **once for every array element**.
 - Does **not return a new array**.
 - Mainly used for displaying data, printing values, or performing actions.
-

Syntax

```
array.forEach(function(element, index, array) {  
    // Code  
});
```

or using an arrow function:

```
array.forEach((element, index) => {  
    // Code  
});
```

Parameters

Parameter Description

element	Current array element
index	Current index (optional)
array	Original array (optional)

Example 1: Print All Elements

```
let numbers = [10, 20, 30, 40];
```

```
numbers.forEach(function(num) {  
  console.log(num);  
});
```

Output

10

20

30

40

Example 2: Using Arrow Function

```
let numbers = [1, 2, 3, 4];
```

```
numbers.forEach(num => console.log(num));
```

Output

1

2

3

4